

PumpScript

Retrospective & Self-Assessment — Part 1

CSE 341 · Concepts of Programming Languages

Gebze Technical University · 8 May 2026

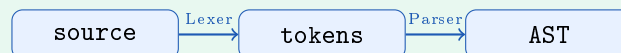
Student Name	Yusuf Eren Nalbant
Student No	220104004049
Language	PumpScript
Deliverable	D5 — Part 1 Retrospective

Retrospective

✓ What is Working

The most successful part of Part 1 was implementing the lexer and parser in Python. Since I already had background knowledge of similar concepts from C, the transition to Python felt natural and more concise. I was able to write cleaner code with less boilerplate, and the token-based structure of the lexer translated directly from what I already understood conceptually.

The implementation passed all test cases successfully. Three valid programs parse without errors and produce correct, readable AST output via the `-dump-ast` flag. Five malformed programs are all rejected with informative error messages that include the exact line number. I am satisfied that the core pipeline —



— is working correctly and consistently for all programs in the test suite.

△ What was Hard

Two things stood out as genuinely difficult during Part 1, at opposite ends of the project.

Choosing the domain. Knowing the domain should be sports was easy, but deciding *what exactly* to do with sports took significant time. The question was: what constructs would make this a real DSL and not just a renamed generic language? I had to ask myself what a fitness coach actually needs to express — exercises, sets, reps, weights, rest periods, routines. Only after thinking through real workout programs did the language design become clear.

The static vs. dynamic scoping decision. I understood both concepts from lectures, but had to think carefully about what would actually break in a workout DSL if dynamic scoping were used. Realising that a caller's parameter could silently override a callee's parameter under dynamic scoping — something a fitness coach would never expect — made the case for static scoping concrete and personally convincing, not just a textbook answer.

→ Plan for Part 2

For Part 2, I need to add a type checker and a full interpreter.

Type checker. I have background in semantics — both static and dynamic — so I feel reasonably confident about implementing the type checker. I understand that it will walk the AST and verify type consistency: for example, that **weight** always receives a **float**, **completed** a **bool**, and that routine call arguments match declared parameter types.

Interpreter. This is the part I am most uncertain about. I have not implemented a tree-walking interpreter before, and I am not yet sure how to correctly manage the environment during routine calls so that static scoping is enforced at runtime — not just described in the specification. My plan is to study how environments (scope chains) work in tree-walking interpreters, build the interpreter incrementally starting from expressions and literals, and then add control structures.

I also need to complete D1 sections §4.4 (Semantics), §4.5 (Type System), §4.7 (Expressions & Assignment), and §4.8 (Design Rationale) for the Part 2 submission.

AI Conversation Reference

The design decisions documented in this retrospective were made through an extended conversation with Claude Sonnet 4.6. The full conversation log is available at:

<https://claude.ai/share/1aa8ac86-c139-49ec-8af7-238a6267a3d6>

Note: Due to Claude.ai's sharing limitations, the shared link may not display the entire conversation. All key prompts, responses, accepted/rejected decisions, and reflections are fully documented in the AI Usage Journal (D4). The implementation files (D2) and design specification (D1) independently verify the decisions made during this session.